Pimpri-Chinchwad Education Trust's
**Pimpri-Chinchwad College of Engineering**
(An Autonomous Institute)
Affiliated to Savitribai Phule Pune University (SPPU)
ISO 21001:2018 Certified by TUV

# Assignment 3

**Name :** Om Rajendra Bharambe
**PRN :** 124B1F001
**Batch :** A1
**Date :** 24/02/2026

## Aim:

All types of Join, Sub-Query and View:

a. Write at least 10 SQL queries for suitable database application using SQL DML statements using joins and subqueries.
b. Write queries using group by and having clause.

## Objectives:

- **To implement complex data retrieval** by utilizing various SQL Join operations (INNER, LEFT, RIGHT) to combine data from multiple related tables within the Airline and Civic Voice systems.
- **To apply advanced query logic** using nested, scalar, and correlated sub-queries along with EXISTS clauses to filter datasets based on dynamic conditions and multi-level relationships.
- **To demonstrate data aggregation and virtualization** by using GROUP BY and HAVING clauses for statistical analysis and creating VIEWS to simplify access to complex, multi-table result sets.

# Case Study: Airline Management System

SQL Queries:

## Part A: Joins, Subqueries, and Views

**1. Inner Join:** Get passenger names along with their flight names and seat numbers to see who is flying on which flight.

```
mysql> SELECT p.pname, f.flight_name, b.seat_no
    -> FROM passenger p
    -> INNER JOIN booking b ON p.passenger_id = b.passenger_id
    -> INNER JOIN flight f ON b.flight_id = f.flight_id;
+----------------+-------------+---------+
| pname          | flight_name | seat_no |
+----------------+-------------+---------+
| Rahul Sharma   | AI101       | 12A     |
| Neha Verma     | IN202       | 14B     |
| Sneha Kulkarni | EM404       | 8D      |
+----------------+-------------+---------+
3 rows in set (0.00 sec)
```

---

**2. Left Join:** Find all passengers and their booking details, including those who are registered but haven't booked a flight yet (like Amit Patil).

```
mysql> SELECT p.pname, b.booking_id, b.seat_no
    -> FROM passenger p
    -> LEFT JOIN booking b ON p.passenger_id = b.passenger_id;
+----------------+------------+---------+
| pname          | booking_id | seat_no |
+----------------+------------+---------+
| Rahul Sharma   |          1 | 12A     |
| Neha Verma     |          2 | 14B     |
| Amit Patil     |       NULL | NULL    |
| Sneha Kulkarni |          4 | 8D      |
+----------------+------------+---------+
4 rows in set (0.00 sec)
```

---

**3. Right Join:** List all flights and the passengers who booked them, ensuring flights with zero bookings (like SJ303) are still displayed.

```
mysql> SELECT f.flight_name, f.source, f.destination, p.pname
    -> FROM booking b
    -> RIGHT JOIN flight f ON b.flight_id = f.flight_id
    -> LEFT JOIN passenger p ON b.passenger_id = p.passenger_id;
+-------------+--------+-------------+----------------+
| flight_name | source | destination | pname          |
+-------------+--------+-------------+----------------+
| AI101       | Mumbai | Delhi       | Rahul Sharma   |
| IN202       | Pune   | Bangalore   | Neha Verma     |
| SJ303       | Delhi  | Goa         | NULL           |
| EM404       | Dubai  | Mumbai      | Sneha Kulkarni |
+-------------+--------+-------------+----------------+
4 rows in set (0.00 sec)
```

---

**4. Subquery (*WHERE* clause):** Filter the flight table to only show flights operated by airlines based in India.

```
mysql> SELECT flight_name, source, destination, price
    -> FROM flight
    -> WHERE airline_id IN (SELECT airline_id FROM airline WHERE country = 'India');
+-------------+--------+-------------+---------+
| flight_name | source | destination | price   |
+-------------+--------+-------------+---------+
| AI101       | Mumbai | Delhi       | 5500.00 |
| IN202       | Pune   | Bangalore   | 4500.00 |
| SJ303       | Delhi  | Goa         | 6000.00 |
+-------------+--------+-------------+---------+
3 rows in set (0.01 sec)
```

---

**5. Subquery (Nested):** Find the names of passengers who booked a flight costing more than ₹5000 by chaining subqueries through the booking and flight tables.

```
mysql> SELECT pname
    -> FROM passenger
    -> WHERE passenger_id IN (
    ->      SELECT passenger_id
    ->      FROM booking
    ->      WHERE flight_id IN (SELECT flight_id FROM flight WHERE price > 5000.00)
    -> );
+----------------+
| pname          |
```

```
+----------------+
| Rahul Sharma   |
| Sneha Kulkarni |
+----------------+
2 rows in set (0.00 sec)
```

---

**6. Subquery (Scalar):** Dynamically find the maximum flight price, and display the passenger(s) who booked that specific flight.

```
mysql> SELECT p.pname, f.flight_name, f.price
    -> FROM passenger p
    -> JOIN booking b ON p.passenger_id = b.passenger_id
    -> JOIN flight f ON b.flight_id = f.flight_id
    -> WHERE f.price = (SELECT MAX(price) FROM flight);
+----------------+-------------+----------+
| pname          | flight_name | price    |
+----------------+-------------+----------+
| Sneha Kulkarni | EM404       | 15000.00 |
+----------------+-------------+----------+
1 row in set (0.01 sec)
```

---

**7. Create a View:** Create a virtual table (BookingItinerary) that joins four tables together, making it easier to run complex queries later without rewriting the joins.

```
mysql> CREATE VIEW BookingItinerary AS
    -> SELECT b.booking_id, p.pname, a.airline_name, f.flight_name, f.source, f.destination,
b.booking_date
    -> FROM booking b
    -> JOIN passenger p ON b.passenger_id = p.passenger_id
    -> JOIN flight f ON b.flight_id = f.flight_id
    -> JOIN airline a ON f.airline_id = a.airline_id;
Query OK, 0 rows affected (0.03 sec)
```

---

**8. Querying the View:** Use the newly created BookingItinerary view to quickly find all travel itineraries departing from or arriving in Mumbai.

```
mysql> SELECT * FROM BookingItinerary
    -> WHERE source = 'Mumbai' OR destination = 'Mumbai';
+------------+----------------+--------------+-------------+--------+-------------+--------------+
| booking_id | pname          | airline_name | flight_name | source | destination | booking_date |
+------------+----------------+--------------+-------------+--------+-------------+--------------+
|          1 | Rahul Sharma   | Air India    | AI101       | Mumbai | Delhi       | 2026-03-04   |
|          4 | Sneha Kulkarni | Emirates     | EM404       | Dubai  | Mumbai      | 2026-03-04   |
```

```
+------------+---------------+-------------+------------+--------+------------+-------------+
2 rows in set (0.00 sec)
```

---

## Part B: GROUP BY and HAVING Clauses

**9. GROUP BY:** Count the total number of distinct flights operated by each airline.

```
mysql> SELECT a.airline_name, COUNT(f.flight_id) AS total_flights
    -> FROM airline a
    -> LEFT JOIN flight f ON a.airline_id = f.airline_id
    -> GROUP BY a.airline_name;
+--------------+---------------+
| airline_name | total_flights |
+--------------+---------------+
| Air India    |             1 |
| Indigo       |             1 |
| SpiceJet     |             1 |
| Emirates     |             1 |
+--------------+---------------+
4 rows in set (0.00 sec)
```

---

**10. GROUP BY with HAVING:** Find airlines whose average flight ticket price is strictly greater than ₹5000.

```
mysql> SELECT a.airline_name, AVG(f.price) AS average_price
    -> FROM airline a
    -> JOIN flight f ON a.airline_id = f.airline_id
    -> GROUP BY a.airline_name
    -> HAVING AVG(f.price) > 5000.00;
+--------------+---------------+
| airline_name | average_price |
+--------------+---------------+
| Air India    |   5500.000000 |
| SpiceJet     |   6000.000000 |
| Emirates     |  15000.000000 |
+--------------+---------------+
3 rows in set (0.00 sec)
```

---

**11. GROUP BY with HAVING:** Find flights that have at least one confirmed booking by grouping the flights and counting the associated booking IDs.

```
mysql> SELECT f.flight_name, COUNT(b.booking_id) AS total_bookings
    -> FROM flight f
    -> LEFT JOIN booking b ON f.flight_id = b.flight_id
    -> GROUP BY f.flight_name
    -> HAVING COUNT(b.booking_id) >= 1;
+-------------+----------------+
| flight_name | total_bookings |
+-------------+----------------+
| AI101       |              1 |
| IN202       |              1 |
| EM404       |              1 |
+-------------+----------------+
3 rows in set (0.00 sec)
```

---

# Case Study: Civic Voice Interface System

SQL Queries:

## Part A: Joins, Subqueries, and Views

**1. Inner Join:** Get the user's basic account name and email alongside their physical location details from their profile.

```
mysql> SELECT u.name, u.email, c.state, c.district
    -> FROM user u
    -> INNER JOIN citizenprofile c ON u.user_id = c.user_id;
+--------------+-----------------------+-------------+-----------+
| name         | email                 | state       | district  |
+--------------+-----------------------+-------------+-----------+
| Rahul Sharma | rahul.sharma@email.com | Maharashtra | Pune      |
| Neha Verma   | neha.verma@email.com   | Delhi       | New Delhi |
| Amit Patil   | amit.patil@email.com   | Maharashtra | Mumbai    |
+--------------+-----------------------+-------------+-----------+
3 rows in set (0.00 sec)
```

---

**2. Left Join:** List all users and any documents they have uploaded, ensuring users who haven't uploaded anything (like Amit Patil) still appear in the results.

```
mysql> SELECT u.name, d.doc_name, d.status
    -> FROM user u
    -> LEFT JOIN userdocument d ON u.user_id = d.user_id;
+--------------+----------------+----------+
| name         | doc_name       | status   |
+--------------+----------------+----------+
| Rahul Sharma | Aadhaar Card   | Approved |
| Rahul Sharma | PAN Card       | Approved |
| Neha Verma   | Driving License | Approved |
| Amit Patil   | NULL           | NULL     |
+--------------+----------------+----------+
4 rows in set (0.00 sec)
```

---

**3. Right Join:** Show all citizen profiles and link them to the user's phone number, prioritizing the profile data.

```
mysql> SELECT c.full_name, c.age, c.gender, u.phone_number
    -> FROM user u
    -> RIGHT JOIN citizenprofile c ON u.user_id = c.user_id;
+----------------+------+--------+--------------+
| full_name      | age  | gender | phone_number |
+----------------+------+--------+--------------+
| Rahul S. Sharma |   28 | M      | 9876543210   |
| Neha Verma     |   25 | F      | 8765432109   |
| Amit Patil     |   32 | M      | 7654321098   |
+----------------+------+--------+--------------+
3 rows in set (0.00 sec)
```

---

**4. Subquery (WHERE clause):** Find the profile details of users who have set their preferred app language to 'Hindi'.

```
mysql> SELECT full_name, state, district
    -> FROM citizenprofile
    -> WHERE user_id IN (SELECT user_id FROM user WHERE language = 'Hindi');
+------------+-------+-----------+
| full_name  | state | district  |
+------------+-------+-----------+
| Neha Verma | Delhi | New Delhi |
+------------+-------+-----------+
1 row in set (0.00 sec)
```

**5. Subquery (Nested):** Find the names and emails of users who have successfully uploaded an 'Aadhaar Card' by linking through the document table.

```
mysql> SELECT name, email
    -> FROM user
    -> WHERE user_id IN (
    ->      SELECT user_id
    ->      FROM userdocument
    ->      WHERE doc_name = 'Aadhaar Card'
    -> );
+--------------+------------------------+
| name         | email                  |
+--------------+------------------------+
| Rahul Sharma | rahul.sharma@email.com |
+--------------+------------------------+
1 row in set (0.00 sec)
```

---

**6. Create a View:** Create a virtual table (ComprehensiveUserView) that joins user account details with their profile information to make frequent queries easier.

```
mysql> CREATE VIEW ComprehensiveUserView AS
    -> SELECT u.user_id, u.name, u.phone_number, u.language, c.state, c.district, c.age
    -> FROM user u
    -> JOIN citizenprofile c ON u.user_id = c.user_id;
Query OK, 0 rows affected (0.02 sec)
```

---

**7. Querying the View:** Use the newly created view to quickly find all users located in 'Maharashtra'.

```
mysql> SELECT * FROM ComprehensiveUserView
    -> WHERE state = 'Maharashtra';
+---------+--------------+--------------+----------+-------------+----------+------+
| user_id | name         | phone_number | language | state       | district | age  |
+---------+--------------+--------------+----------+-------------+----------+------+
|       1 | Rahul Sharma | 9876543210   | English  | Maharashtra | Pune     |   28 |
|       3 | Amit Patil   | 7654321098   | Marathi  | Maharashtra | Mumbai   |   32 |
+---------+--------------+--------------+----------+-------------+----------+------+
2 rows in set (0.00 sec)
```

---

**8. Multiple Joins:** Link three tables together to see a user's name, their state, and the specific documents they have verified.

```
mysql> SELECT u.name, c.state, d.doc_name
    -> FROM user u
    -> JOIN citizenprofile c ON u.user_id = c.user_id
    -> JOIN userdocument d ON u.user_id = d.user_id;
+--------------+-------------+-----------------+
| name         | state       | doc_name        |
+--------------+-------------+-----------------+
| Rahul Sharma | Maharashtra | Aadhaar Card    |
| Rahul Sharma | Maharashtra | PAN Card        |
| Neha Verma   | Delhi       | Driving License |
+--------------+-------------+-----------------+
3 rows in set (0.00 sec)
```

---

**9. Subquery with EXISTS:** Find the names of users who have at least one approved document in the system.

```
mysql> SELECT name
    -> FROM user u
    -> WHERE EXISTS (
    ->      SELECT 1
    ->      FROM userdocument d
    ->      WHERE d.user_id = u.user_id AND d.status = 'Approved'
    -> );
+--------------+
| name         |
+--------------+
| Rahul Sharma |
| Neha Verma   |
+--------------+
2 rows in set (0.00 sec)
```

---

## Part B: GROUP BY and HAVING Clauses

**10. GROUP BY:** Count the total number of registered citizen profiles grouped by their state.

```
mysql> SELECT state, COUNT(profile_id) AS total_citizens
    -> FROM citizenprofile
    -> GROUP BY state;
+-------------+----------------+
| state       | total_citizens |
+-------------+----------------+
| Maharashtra |              2 |
| Delhi       |              1 |
+-------------+----------------+
2 rows in set (0.00 sec)
```

---

**11. GROUP BY with HAVING:** Find states that have strictly more than 1 registered user.

```
mysql> SELECT state, COUNT(profile_id) AS total_citizens
    -> FROM citizenprofile
    -> GROUP BY state
    -> HAVING COUNT(profile_id) > 1;
+-------------+----------------+
| state       | total_citizens |
+-------------+----------------+
| Maharashtra |              2 |
+-------------+----------------+
1 row in set (0.00 sec)
```

---

**12. GROUP BY and Joins:** Count how many documents each user has uploaded, grouping the results by the user's name.

```
mysql> SELECT u.name, COUNT(d.doc_id) AS total_documents
    -> FROM user u
    -> LEFT JOIN userdocument d ON u.user_id = d.user_id
    -> GROUP BY u.name;
+--------------+-----------------+
| name         | total_documents |
+--------------+-----------------+
| Rahul Sharma |               2 |
| Neha Verma   |               1 |
| Amit Patil   |               0 |
+--------------+-----------------+
3 rows in set (0.00 sec)
```

---

**13. GROUP BY with HAVING and Joins:** Find users who have uploaded 2 or more documents.

```
mysql> SELECT u.name, COUNT(d.doc_id) AS total_documents
    -> FROM user u
    -> JOIN userdocument d ON u.user_id = d.user_id
    -> GROUP BY u.name
    -> HAVING COUNT(d.doc_id) >= 2;
+--------------+-----------------+
| name         | total_documents |
+--------------+-----------------+
| Rahul Sharma |               2 |
+--------------+-----------------+
1 row in set (0.00 sec)
```

---

# Conclusion:

The successful completion of this assignment highlights the power of advanced SQL techniques in managing complex relational data. By implementing various **Joins (Inner, Left, Right)**, the project effectively demonstrated how to synthesize information from fragmented tables—such as linking passengers to their specific flight itineraries or users to their uploaded documents. The use of **Sub-queries** and the **EXISTS** clause provided a mechanism for sophisticated, multi-layered data filtering that standard queries cannot achieve alone. Additionally, the creation of **Views** showcased a practical approach to data virtualization, simplifying access to frequent, high-complexity result sets. Finally, the application of **GROUP BY** and **HAVING** clauses enabled meaningful statistical analysis, allowing for the aggregation of data to uncover trends like airline pricing averages and regional user distributions. These skills are fundamental for developing robust, scalable backend systems like the **Airline Management System** and the **Civic Voice Interface**.